The Published P-256 Seed Was Not the First Viable Candidate: A Forensic Audit of NIST Curve Generation

Prabhu Rangaraju

***Abstract*—Anthropic and Valsorda reported that P-256's published seed (candidate 1,842) was preceded by a viable candidate 876; we independently verify this skip in a preregistered audit, strengthening it to the strictest criteria. We verify the seed-to-curve derivation pipeline; other runs remain unevaluated.**

***Index Terms*—elliptic-curve cryptography, NIST curves, P-256, verifiable randomness, forensic audit, cryptographic standards.**

Introduction

For more than two decades, the seeds behind the NIST elliptic curves arrived with no explanation of where they came from. These curves—P-192, P-224, P-256, P-384, P-521—underpin an enormous share of the world's encrypted traffic, from web TLS to messaging protocols, yet the random seeds from which their parameters were derived were published without any account of their origin. The vacuum attracted speculation for years, most colorfully a 2015 recollection by Jerry Solinas, a longtime NSA employee, of a hashed humorous passphrase along the lines of "give Bob and Jerry a raise"—a lead that sent several serious searches down the wrong path. Cryptographer Filippo Valsorda eventually put up a $36,864 bounty for anyone who could solve the mystery.

In October 2026, researchers at Anthropic claimed it. An internal AI model, asked cold to work out how the seeds were generated, traced all five NIST prime-curve seeds in under half an hour to an unlikely source: the `lrand48` pseudorandom generator shipped with 1990s Unix C libraries, each invocation seeded with a Unix timestamp [3]. The team donated the bounty to the Internet Security Research Group. The finding was reassuring in one sense—the seeds came from a mundane, reproducible process—but it opened a sharper question. Knowing *how* the candidates were produced does not tell us *which* candidate was chosen, or why.

That selection question is the subject of this article. Anthropic's reconstruction showed that the published seeds were not always the first viable candidates of their runs: the published P-256 seed is candidate 1,842 of its run, while candidate 876 was also viable [3]. If the generation process simply took the first candidate that yielded a suitable curve, candidate 876—not 1,842—would have been published. It was not. Something, whether mundane or deliberate, selected among viable candidates, and nobody knows what. One candidate-rejection rule—a bit-length rejection reported independently by Valsorda [11]—has been identified; the rest of the selection process remains unresolved.

Why does a skip matter? Verifiable randomness means the seed commits to the curve with no room for choice. A documented selection rule restores that; an undocumented skip reopens exactly the room for hidden selection the design was meant to close—this is where a weakness could be introduced without detection. That is what we tested.

We report an independent, preregistered forensic audit of that selection. We reconstructed the full seed-to-curve pipeline from the published standards, verified it byte-for-byte against every published prime-curve selection, and tested the first-viable hypothesis directly by computing curve orders. Our headline result is a verified refutation: for P-256, the published seed was *not* the first viable candidate. Candidate 876 has prime order, passes even the strictest documented strength criteria, and precedes the published candidate—yet was passed over.

We are careful about what this does and does not establish. A refuted first-viable rule is not evidence of a backdoor, and a verified mundane process is not proof that nothing was selected. Our audit ran only part of its preregistered scope—compute limits bound what we could enumerate—so we report exactly what we verified, mark everything else unevaluated, and name the selection rule what it is: unidentified. Beyond the verified result, the work offers a reusable template for future audits: how to audit a verifiable-randomness claim, preregistered, with every negative pre-committed.

Background: Seeds Without a Story

Start with what a seed does. In the FIPS 186-2 generation method [1], a seed is the input to a deterministic recipe: expand it with SHA-1 (appendix A.6.4), derive the curve coefficient b, check the result against a verification equation $b^2c \equiv -27 \pmod{p}$ (A.6.5), and accept the curve only if its order—the number of points on it—has the required form (a large prime factor, small cofactor). The seed is the only arbitrary input. Everything else follows mathematically. That is the point of the design: anyone can re-derive the curve from the seed and confirm nothing was tuned. But the design assumes the seed itself is above suspicion—and for 25 years, nobody could say where the seeds came from.

Anthropic's October 2026 report closed the origin question with an anticlimax cryptographers welcomed: no hashes, no passphrases. Each seed is 20 bytes drawn as twenty `lrand48()` outputs shifted right by 23 bits, from a generator seeded once with a Unix timestamp; candidates are numbered

sequentially, twenty bytes at a time [3]. The detective work behind that finding is worth recounting, because it sets the forensic bar. The model wrote a scanner that slid along each published seed bit by bit, checking whether adjacent chunks matched consecutive outputs of 34 combinations of generators and output slices from the 1990s. The telltale was a pattern of zero top bits across all twenty words in several seeds—`lrand48` returns only the top 31 bits of its 48-bit state, so the missing bit is a fingerprint. Recovering the prime-curve seeds then required solving a lattice problem with the LLL algorithm, because those seeds keep only a few bits of each generator output; every recovered 48-bit state regenerates all 160 bits of its seed, which does not happen by chance. The five NIST prime curves map to timestamps between May 1997 and April 1998, and five ANSI X9.62 example curves match the same pattern [2], [3]. The base points of P-192 and the X9.62 curves were traced separately to a generator inside FreeLIP, Arjen Lenstra's long-integer package, seeded with small integers such as 20000 and 20517—a different mechanism, consistent with a separate session and a different author of that step.

But origin is not selection. Anthropic's inventory records skipped viable candidates: P-256's published seed is candidate 1,842 while candidate 876 was viable; the P-224 run skipped three viable candidates; the X9.62 runs skipped seven [3]. Independently, Filippo Valsorda reproduced the generation process and reported two additional undocumented criteria beyond X9.62—one a usable bit-length rejection rule, one still unexplained (hence not a usable rule)—leaving roughly twice as many valid seeds as were published [11]. This is the gap our audit addresses: given the verified generation process, was the published seed simply the first viable candidate? The question is falsifiable—recompute the orders, check the numbering—and it is the exact place a hidden selection, innocent or otherwise, would leave a trace.

The question matters beyond historical curiosity. Trust in shared parameters rests on verifiable randomness—the idea, championed by the SafeCurves project [8], that parameters should be generated by a documented, reproducible process with no room for hidden selection. The history here is instructive. When the NIST curves' provenance was opaque, the community split: some accepted the parameters on authority, while others—notably the designers of the Brainpool curves—insisted on rigid, fully documented generation precisely to eliminate the selection step this article audits. The Dual EC episode showed what opacity costs at its worst: when a standard's generation cannot be audited, suspicion fills the gap whether or not any weakness exists, and the practical exploitation of Dual EC retroactively poisoned trust in the process [9]. Our finding sits between these poles: a verified mundane process with one unexplained selection. It is neither an exoneration nor an accusation, and the article is written to keep it that way. As cryptography transitions to post-quantum standards—where FIPS 203 already standardizes new parameters the world must trust [10]—the community needs working templates for auditing verifiable-randomness claims, including ones that report honest partial results.

METHOD: A PREREGISTERED FORENSIC AUDIT

Forensic claims are easy to make and hard to check, which is why we preregistered the audit before executing it. Hypotheses, search space, viability definitions, controls, stop rules, and a compute cap were frozen in a preregistration document that survived two rounds of independent methods review, each by a fresh panel that saw only the proposal. Preregistration is unusual in this kind of work; we adopted it because the failure mode it guards against—reading the answer off the data and then describing the method as if it had been planned—is precisely what makes forensic findings untrustworthy. A forensic audit without preregistration can always be re-tuned, consciously or not, until the data confesses; the frozen document is what lets a reader distinguish a test from a story. Every deviation below was disclosed rather than folded in silently, and every negative outcome had a pre-committed interpretation, including "selection rule unidentified."

Pipeline reconstruction. For each candidate n in a run, we regenerate the 20-byte seed from the `lrand48` stream (`srand48`(t), twenty `lrand48()`$\gg$23 draws), derive b via FIPS 186-2 A.6.4, and verify via A.6.5. One correction emerged during construction: a square-root shortcut valid only for primes $p \equiv 3 \pmod 4$ fails on P-224's prime $(1 \bmod 4)$; we replaced it with the general Tonelli–Shanks method [5]. Finding and fixing that bug before any measurement is exactly the kind of silent assumption that voids a forensic claim if left in place. Curve orders were computed with PARI/GP's `ellcard`, cross-checked with `ellsea` [4]; the two algorithms must agree before a candidate enters any count.

Viability tiers. In plain terms: the order is the number of points on the curve; the cofactor h measures how far that count is from being prime ($h = 1$ means the order itself is prime, which the standards required); and the embedding degree guards against a known class of attacks that reduce the discrete-log problem to a weaker setting. A candidate is viable under `V_loose` if its order $N = h \cdot q$ with q prime, $q \neq p$ (excluding anomalous curves, whose order equals the field prime), and cofactor $h \leq 8$ (an arbitrary loose bound). The strict tier `V_strict` pins $h = 1$ and requires embedding degree $k > 20$ (the MOV condition [6])—our conservative superset of the standards' requirements, stricter than anything the 1997 process is documented to have applied. Primality was tested under a five-variant Miller–Rabin panel with fixed bases (reproducible, no random rounds) [7]: bases $\{2\}$ (1 pass), $\{2, 3\}$ (5 passes), $\{2, 3, 5, 7, 11\}$ (10 passes), trial division to 10^3 then MR$\times$10, and trial division to 10^6 then MR$\times$10; a candidate counts as skipped-viable only if its order tests prime under all five variants. The tiers matter because "viable" is doing load-bearing work in the argument: a skip is only interesting if the skipped candidate genuinely qualified.

Controls. Three controls validate the machinery, and each guards a different failure mode. First, a composite-order control curve, built deliberately over P-192's field, is correctly rejected at the viability stage (its order fails the primality requirement)—this checks that the viability test is not a rubber stamp that would pass anything handed to it. Second, the documented orders of published curves recompute byte-identically

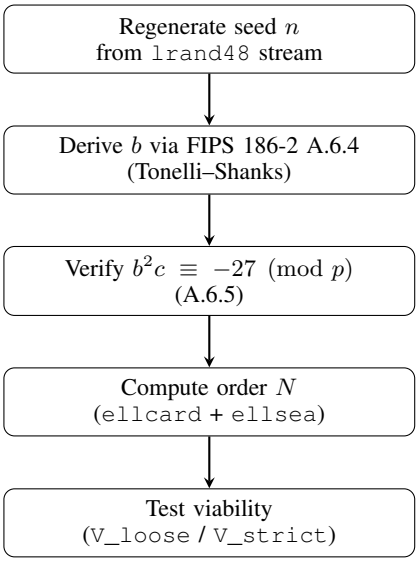


Fig. 1. The reconstructed seed-to-viability pipeline. Each stage was verified independently before the next was trusted.

(P-256's published order matches FIPS 186-2 to the digit), anchoring our arithmetic to ground truth rather than to our own code's say-so. Third, every PRNG implementation was validated against published reference vectors (e.g., glibc manual test vectors), and the binary-curve search machinery was calibrated by recovering known seeds (see the binary-curve calibration below)—so a future null result means something about the world, not about a bug. A 100-CPU-hour cap (a long weekend on our workstation) bound total compute; candidates not reached are reported unevaluated, never assumed. Figure 1 summarizes the pipeline.

Result I: The P-256 Refutation

The first-viable hypothesis fails for P-256, verified end to end. Candidate 876 of the P-256 run has the order, cofactor, and primality properties shown in Table 1: order N prime under all five primality variants, cofactor $h = 1$, $q = N \neq p$ (non-anomalous), all quantities 256 bits. Candidate 876 passed the A.6.5 seed-to-curve verification ($b^2c \equiv -27$), confirming it is a genuine run candidate rather than an unanchored seed. Its order was computed by two distinct algorithms within PARI/GP (`ellcard` and `ellsea`) in agreement; the independent anchor is the byte-identical match of the published candidate's recomputed order to the documented NIST value. The published candidate, $n = 1842$, has order byte-identical to the documented NIST value in FIPS 186-2 [1]. Since $876 < 1842$ and both are viable under the preregistered tiers—preceding assumes the validated `lrand48` stream model extends unchanged to candidate 876—the published seed was not the first viable candidate. The refutation is deterministic: two order computations suffice, and both are replayable from the archived materials. (Anthropic's inventory reported this skip; our contribution is its independent, preregistered verification and the reusable audit template.)

The cheapest innocent explanation does not rescue the hypothesis. Candidate 876 passes even `V_strict`: $h = 1$, and its embedding degree exceeds 20 ($p^d \not\equiv 1 \pmod{q}$ for all $1 \leq d \leq 20$, hence $k > 20$ by the MOV threshold). Candidate 876 also passes Valsorda's independently reported bit-length rejection rule ($\mathrm{bitlen}(N_{876}) = 256 = \mathrm{bitlen}(p)$, so it is not rejected): the skip is not explained by that filter either. Whatever excluded candidate 876, it was not the order, cofactor, or embedding-degree requirements as stated in the standards, nor the one independently documented extra filter. (A period primality test could still have failed 876 by weakness or by chance—a strength-criteria-adjacent explanation the unrun mundane-rule battery is meant to probe.) Under our preregistered outcome table, the track now proceeds to a battery of mundane selection rules (ordering permutations, period-plausible primality variants, batch-then-pick rules); if none reproduces the published selection, the pre-committed outcome is "selection rule unidentified"—a checkable negative, not a failure.

TABLE I
Viability tuple for P-256 candidate 876 (the skipped candidate) versus the published candidate 1842.

Property	$n = 876$ (skipped)	$n = 1842$ (published)
Order N prime	Yes (5/5 variants)	Yes (5/5 variants)
Cofactor h	1	1
$q = N/h \neq p$	Yes (non-anomalous)	Yes (non-anomalous)
Bit length	256	256
Orders distinct, both prime	Yes	
Matches documented q	n/a (unpublished)	Byte-identical

What could the rule have been? The honest answer is that several mundane mechanisms fit the evidence so far, and the audit was designed to distinguish them rather than to presume. The implementer may not have tested candidates in generation order—a 1997 C program scanning for the first viable curve might equally have scanned backward, or strided, or raced parallel workers and taken the first finisher. The viability test actually used may have been weaker than ours: a candidate we call viable under a strong modern test could have failed the period test, or been too slow to evaluate on 1997 hardware and skipped as unevaluable. Or the rule may have been batch-then-pick: generate a batch, then take the last viable, or the one with the extremal seed. Or—the possibility the preregistration explicitly refuses to exclude—a human may simply have picked. Each of these is a concrete, testable hypothesis in the battery described in the next section; none is assumed. The point worth underscoring is that "unidentified" is not a euphemism for "suspicious": most mundane rules, if confirmed, would close the question as decisively as a backdoor finding would open it.

Result II: The Pipeline Verifies

Before the refutation means anything, the reconstruction itself must be trustworthy. It is. For all ten published prime-curve selections—the five NIST curves and five X9.62 example curves—the seed-to-b derivation reproduces the published coefficient byte-exactly: nine under the principal Tonelli–Shanks root, and P-224 under the negated root (both roots satisfy the A.6.5 verification equation, so this is a choice

TABLE II
BINARY-CURVE CALIBRATION: ALL FOUR EXPLAINED SEEDS RECOVERED AT HALVED-CLOCK READINGS OF A SINGLE 1998 SESSION.

Seed	t_{half}	Candidate n	Implied UTC
B-233	446413329	155	1998-04-17 15:24:18
B-283	446413432	32	1998-04-17 15:27:44
B-409	446413509	60	1998-04-17 15:30:18
B-571	446413564	73	1998-04-17 15:32:08

of representative, documented per run). The published candidate indices (P-192: 139; P-224: 7,026; P-256: 1,842; P-384: 2,688; P-521: 1,136; prime192v2: 6,013; prime192v3: 5,228; prime239v1: 2,185; prime239v2: 2,320; prime239v3: 3,048) each regenerate their published seed and coefficient through the single pipeline. The byte-exact derived-b equals published-b match is the strongest possible check that the deterministic seed-to-coefficient derivation was rebuilt correctly (the PRNG/timestamp reconstruction itself is Anthropic's, cited and reused): ten independent reproductions, zero mismatches.

RESULT III: A CONTROL—THE MACHINERY FINDS WHAT IT SHOULD

Before trusting the P-256 refutation, we need to know the search machinery isn't hallucinating—so we ran it on the four binary-curve seeds whose provenance is already explained. For the binary curves, the generator was seeded not with a full Unix timestamp but with the timestamp divided by two (a "halved clock," a detail of the binary-curve tooling)—so we searched a grid of halved-clock values, each of which converts to a UTC time by doubling. As a control on the machinery, we recovered all four explained binary seeds (B-233, B-283, B-409, B-571) at halved-clock readings of a single 1998 session. The recovered generator states (halved-clock values 446413329, 446413432, 446413509, and 446413564, at candidate positions 155, 32, 60, and 73) span 15:24 to 15:32 UTC on 17 April 1998—for example, $446413329 \times 2 = 892826658$ epoch seconds is 15:24:18 UTC. Each curve came from its own invocation with its own clock reading, minutes apart, as if someone ran the generator four times that afternoon—consistent with the halved-clock mechanism, and strong evidence the machinery recovers real historical sessions rather than coincidences.

That the blind inversion recovered all four—each at a distinct and plausible reading—validates the search machinery: a future null result will be interpretable as evidence about the world, rather than a methods artifact.

NEXT: THE MUNDANE-RULE BATTERY

The preregistration does not stop at refutation. Before any selection among viable candidates can be called unexplained, the audit must test the mundane rules a 1997 implementer might have followed—each pre-committed, each with a precise match criterion (a rule explains the data only if it reproduces the published selection in every run with skips). The battery includes ordering permutations (a C program need not test candidates in generation order—forward, reverse, and strided orders are all tested); period-plausible primality variants (a candidate viable under a modern strong test may have failed the weaker test actually used in 1997, and vice versa); batch-then-pick deterministic rules (last viable, extremal seed value); a genuine wall-clock race model (identical machines, measured per-candidate evaluation costs, earliest cumulative completion wins—a workstation farm in 1997 is more principled than any ordering rule); and operational hypotheses (crash-and-restart with a fresh timestamp, interrupted runs, staged viability checks). A synthetic control battery—four simulated runs with planted selection rules, one with an unlisted rule—would validate that the machinery can both recover a known rule and honestly report "unidentified" when none matches. None of this battery has been run; its compute fits the preregistered cap for the sub-384-bit runs. We describe it here because the refutation's meaning depends on it: the P-256 skip is a verified fact, but whether any mundane rule explains it is the open question the battery was built to answer.

SCOPE AND LIMITS

Honesty about scope is part of the result. Only the P-256 targeted verification completed: other prime runs were not enumerated, because full enumeration needs roughly 200–300 CPU-hours on our hardware—well beyond the preregistered cap. (An early projection of 100–130 CPU-hours turned out to conflate wall-clock throughput with CPU-hours and to ignore bit-size scaling; the corrected accounting is part of the archived record.) The eight sub-384-bit runs fit the cap at an estimated 55–60 CPU-hours and are the natural next step. The B-163 grid was calibrated but not run. The composite control and documented-order checks cover 3 of 14 published curves. Everything not verified is marked unevaluated in the report, not assumed—a discipline the preregistration enforced by making "indeterminate" a first-class outcome.

Three hedges bind every claim in this article. First, a refuted first-viable rule is not evidence of a backdoor: selection among viable candidates, by whatever rule, does not imply a weak curve was chosen, and no weakness in P-256 is claimed or implied here. The Dual EC history makes this hedge load-bearing rather than boilerplate: the lesson of that episode is that opacity breeds suspicion, not that every anomaly hides an attack. Second, a verified mundane process is not proof that nothing was selected: our audit cannot see unrecorded human choices, and we do not pretend otherwise. Our order computations cover the P-256 run only; the skip inventories for P-224 and the X9.62 runs are Anthropic's, which we cite but did not independently re-derive, and any claim about those runs awaits the enumeration sprint. The run timestamp and candidate-indexing convention for all runs, including P-256's, follow Anthropic's reconstruction; our verification is conditional on those inputs—we verify that (t, n) regenerates the published seed and coefficient, not the timestamp recovery itself. Third, inconclusive results are never framed as insinuation—where the evidence ends, the article says so, in the exact words committed before the data arrived. A forensic audit that cannot say "we do not know" is not an audit but an advocacy piece, and the preregistration's pre-committed negatives exist precisely to keep this one honest.

Discussion: A Template for Verifiable Randomness

The P-256 refutation is a small, solid fact: one viable candidate was passed over, and the strictest documented criteria do not explain why. Its significance lies less in the single skip than in what the audit demonstrates about how such claims should be checked. Verifiable randomness is a claim about a *process*, and processes can be audited—but only if the audit itself is held to account. Our preregistration, with its frozen viability definitions, pre-committed negatives, and compute cap, is what lets a partial result stay honest: the reader can see exactly what was tested, what was not, and what each outcome was allowed to mean before the data arrived.

Three practices generalize beyond this case. First, reproduce the derivation; do not trust the description. Our byte-exact seed-to-coefficient matches across ten independent selections are what make the refutation credible—without them, the order computations would be arithmetic about an unverified pipeline. Second, pin the pipeline's edge cases explicitly. The $p \equiv 3 \pmod 4$ shortcut that broke on P-224 is exactly the kind of silent assumption that voids a forensic claim; we found it only because the preregistration required the pipeline to verify against *every* published selection, not just the convenient ones. Third, give null results a pre-committed home. "Selection rule unidentified" is a publishable, checkable outcome—it tells the next investigator where the boundary of knowledge lies—whereas an unplanned null invites exactly the insinuation the hedges forbid. A fourth practice underwrites the other three: independent verification of the verifier. Our pipeline was checked against ten independent reproductions and two order-computation algorithms precisely because a forensic tool that cannot catch its own errors—like the square-root shortcut we found and fixed—will certify whatever it was built to find. Trust in the audit, like trust in the parameters, has to be earned twice: once in the method, once in the checking of the method.

For the post-quantum transition, the lesson is concrete. New standards will again ask the world to trust generated parameters, and the generation ceremonies should be documented to the level this audit required: enumerated candidate spaces, pinned derivation code, published seeds *and* the rule that selected them, with the selection step logged rather than remembered. Had the 1997 process recorded which rule chose among viable candidates, there would be no paper to write; the audit exists because the log does not. Concretely, a ceremony should publish the candidate-generation program and its exact inputs, the viability predicate applied to each candidate, the deterministic selection rule over the viable set, and the full ordered list of candidates considered—not just the winner. The ceremony itself should be performed or witnessed by multiple independent parties, with each party's transcript published. Future ceremonies should be designed so that the next audit confirms rather than discovers—verifiable randomness, auditable by construction. The marginal cost at generation time is negligible—a log file and a published script—while the forensic cost of its absence, as this case shows, is measured in decades of speculation, a five-figure bounty, and an AI model spending its half-hour on a mystery that a README could have prevented. The standards of tomorrow will be judged, in part, by how little forensics they require.

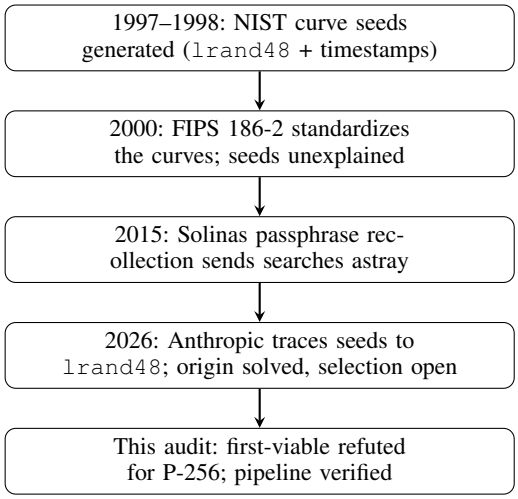


Fig. 2. Twenty-five years from unexplained seeds to a verified refutation. Each step narrowed what the next had to explain.

Conclusion

The verified core—a refuted first-viable rule for P-256, a byte-exact reconstruction pipeline covering ten published selections, and a calibrated binary-curve control—is complete and archived with reproduction materials. The natural next steps are the preregistered ones: the sub-384-bit enumeration sprint that fits the compute cap, the mundane-rule battery on the P-256 run, and the B-163 hypothesis grid. Beyond this case, the work needed is institutional rather than technical: make verifiable randomness auditable by construction, so the next forensic audit of a standards ceremony confirms the record instead of reconstructing it. Standards bodies should require, as part of any parameter-generation submission, the four artifacts this audit had to reverse-engineer: the candidate-generation program with exact inputs, the viability predicate, the deterministic selection rule, and the ordered candidate list. Reviewers should check the selection step with the same seriousness they check the mathematics. That is the future this article argues for, and the preregistered template—frozen definitions, pre-committed negatives, honest partial reporting—is the contribution we offer toward it.

Data and Code Availability

The preregistration document (frozen before any measurement, surviving two rounds of independent methods review), the analysis code, the frozen data files, and a one-command verifier for the headline numbers are archived in the reproduction package accompanying this article. All quantitative claims are recomputable from the archived materials.

Acknowledgments

The author used the AI tool Muse (Muse Spark 1.3, Meta) for drafting, grammatical editing, and code validation of this article, specifically in the Introduction, Background, and Discussion sections. The contents are reviewed, edited, and attested for accuracy by the author.

REFERENCES

[1] National Institute of Standards and Technology, "Digital Signature Standard (DSS)," FIPS PUB 186-2, Jan. 2000.

[2] American National Standards Institute, "Public Key Cryptography for the Financial Services Industry: The Elliptic Curve Digital Signature Algorithm (ECDSA)," ANSI X9.62-1998.

[3] Anthropic, "Claude reveals ECDSA curve origins," https://www.anthropic.com/research/ecdsa-curve-origins, Oct. 2026.

[4] The PARI Group, "PARI/GP version 2.15.4," Univ. of Bordeaux, https://pari.math.u-bordeaux.fr/.

[5] D. Shanks, "Five number-theoretic algorithms," in *Proc. 2nd Manitoba Conf. Numerical Math.*, 1972, pp. 51–70.

[6] A. Menezes, T. Okamoto, and S. Vanstone, "Reducing elliptic curve logarithms to logarithms in a finite field," *IEEE Trans. Inf. Theory*, vol. 39, no. 5, pp. 1639–1646, 1993.

[7] M. O. Rabin, "Probabilistic algorithm for testing primality," *J. Number Theory*, vol. 12, no. 1, pp. 108–126, 1980.

[8] D. J. Bernstein and T. Lange, "SafeCurves: choosing safe curves for elliptic-curve cryptography," https://safecurves.cr.yp.to/.

[9] S. Checkoway *et al.*, "On the practical exploitability of Dual EC in TLS implementations," in *Proc. USENIX Security*, 2014.

[10] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, Aug. 2024.

[11] F. Valsorda, "The NIST Elliptic Curves Seeds Bounty Has Been Claimed," https://words.filippo.io/seeds-solved/, Oct. 2026.

ABOUT THE AUTHOR

Prabhu Rangaraju is a data scientist working on independent audits of machine-learning and cryptographic systems, with a focus on reproducing and stress-testing published empirical claims. His research interests include empirical validation of published claims, verifiable randomness, and measurement methodology for AI systems. He received his M.S. in Information Systems, with a minor in Statistics, from Iowa State University. He is a member of the IEEE. Contact him at the address listed above.